



Sunbeam Institute of Information Technology
Pune and Karad

Operating System Concepts and Linux Programming

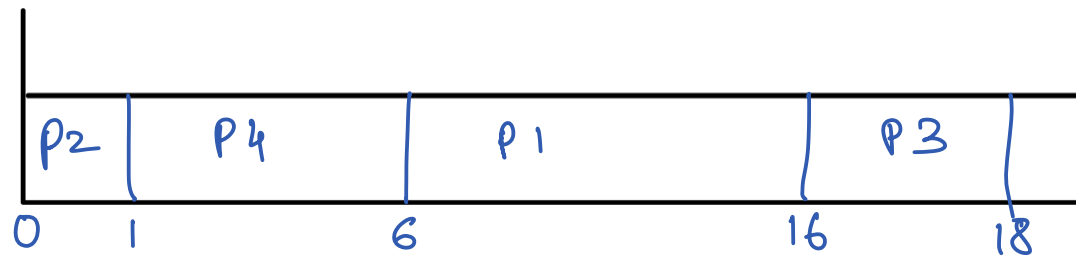
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Priority

(Non preemptive)

Process	Arrival	CPU Burst	Priority
P1	0	10	3
P2	0	1	1 (H)
P3	0	2	4 (L)
P4	0	5	2



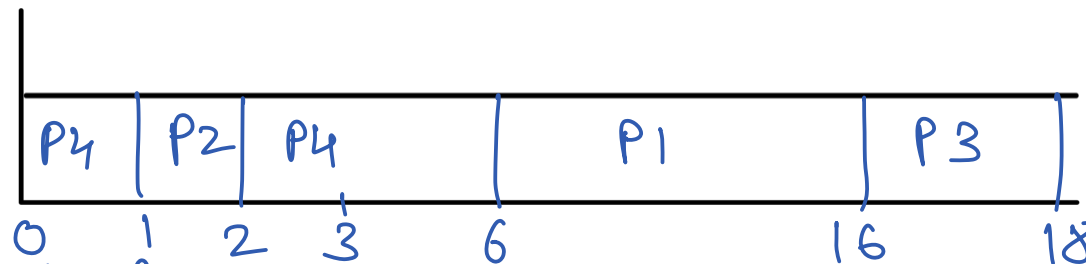
0
P1
P2
P3
P4

P1 (5)
P2 (7)
P3 (4)
P4 (3)
P5 (6)
P6 (3)
P7 (4)

P1
P4
P3
P6
P7
P5
P2

(Preemptive)

Process	Arrival	CPU Burst	Priority
P1	0	10	3
P2	1	1	1
P3	3	2	4
P4	0	5	2



0
P1
P4

starvation:

- due less priority process do not get chance to execute on CPU for longer duration.

Aging:

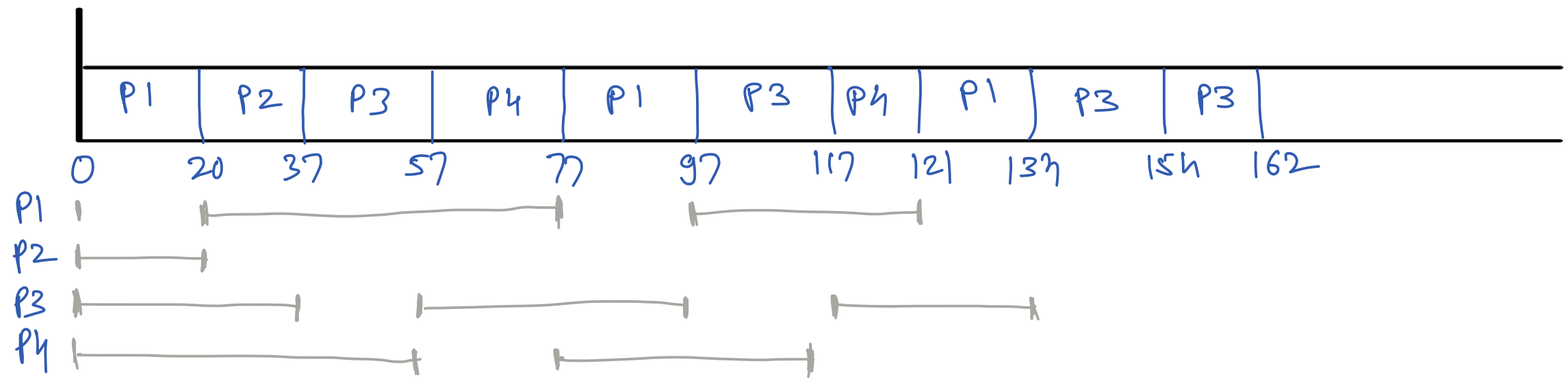
- priority of a process is increased gradually

RR (Round Robin)

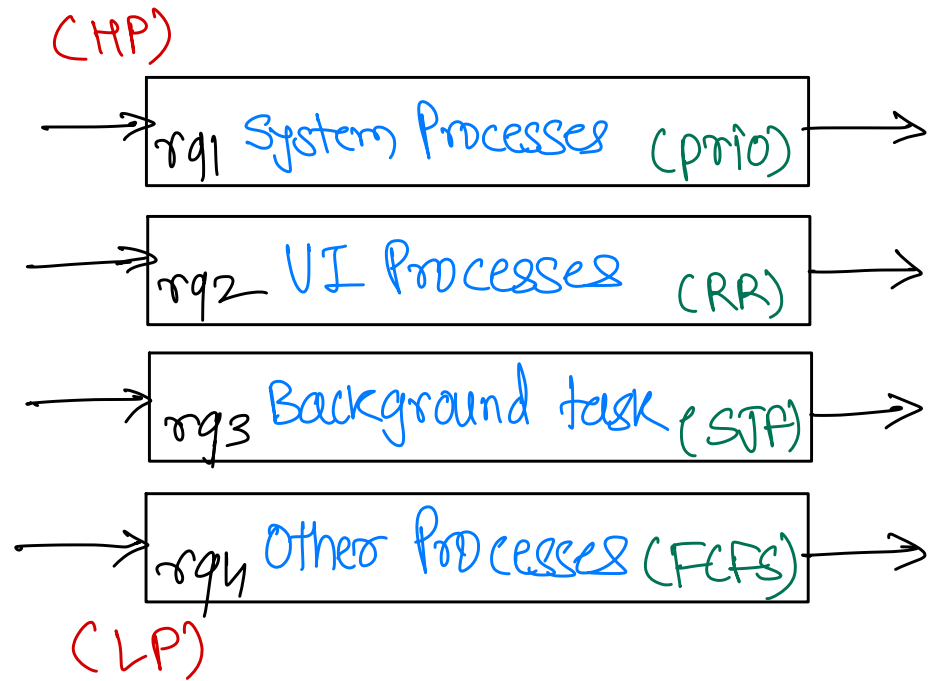
CPU time is divided into time slices called as time quantum
 $TQ = 20$ units

Process	CPU Burst
P1	53
P2	17
P3	68
P4	24

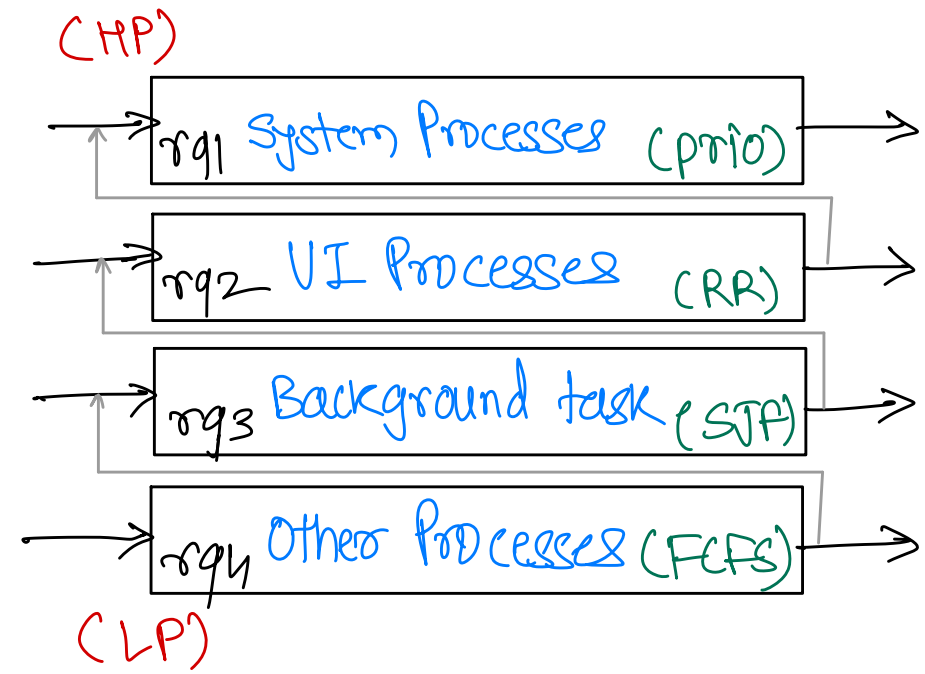
	WT	RT
P1	33, 13	0
P2		20
P3	48, 28, 8	37
P4	4	57



Multi Level Ready Queue



Multi level



Multi Level Feedback

Linux scheduling policies

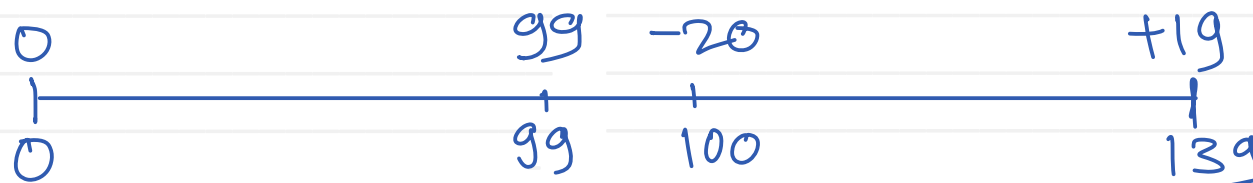
1. Real time
(high priority process never waits for low priority)

2. Non real time
(high priority process may need to wait for low process)

Process classes in Linux

1. SCHED_FIFO (preemptive FFS) 0 - 99
2. SCHED_RR (RR)

3. SCHED_OTHER
4. SCHED_BATCH
5. SCHED_IDLE } (CFS) -20 to +19



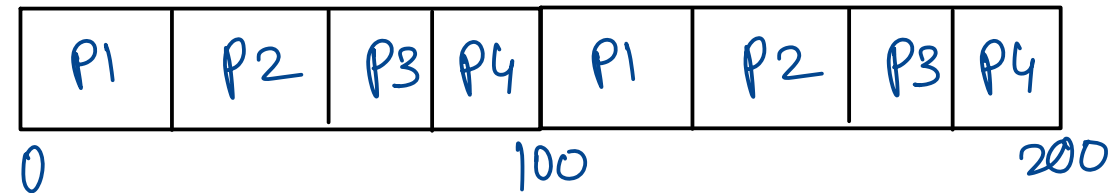
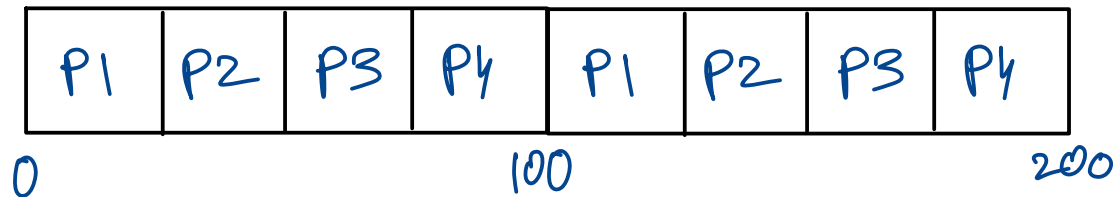
Fair Share (CFS)

- CPU time is divided into time slices (epoch)
- some share of each epoch is given to the processes which are in ready queue.
- share is given to the process on the basis of their priority
- priority of every process is decided by its nice value
- nice values range ---> -20 to +19 (40 values)
 - * -20 - highest priority
 - * +19 - lowest priority

Process	Nice Value
P1	10
P2	10
P3	10
P4	10

Epoch - 100

Process	Nice Value
P1	5
P2	5
P3	10
P4	10



~~100~~
~~101~~
99

SharedVar

thread1() {

P(bs)

sharedVar++;

V(bs)

}

① | (a) read (100)
↓ | (b) inc (101)
③ | (c) write (101)

thread2() {

P(bs)

sharedVar--;

V(bs)

}

② | (a) read (100)
↓ | (b) dec (99)
④ | (c) write (99)

1

bs

- semaphore is internally a counter
- On semaphore two operations can be performed.

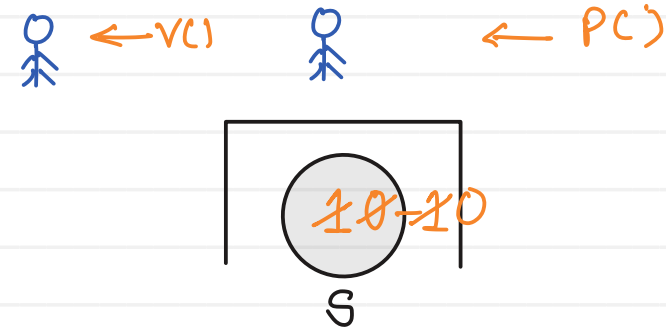
Dec op / Wait() / P() :

- performed before using the resource
- step1 : dec counter
 step2 : if counter < 0 ,
 then block current process

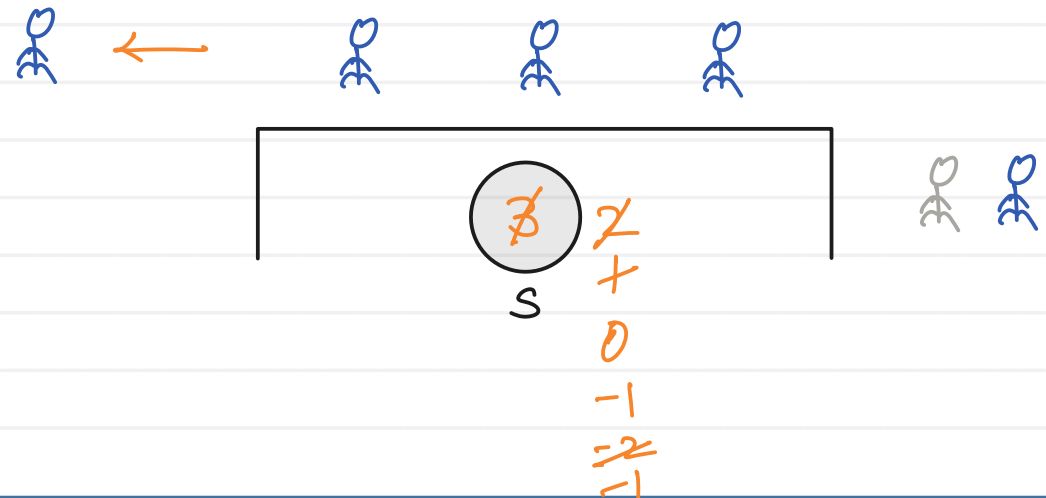
Inc op / post() / signal() / V() ;

- performed when resource is released
- step1 : inc counter
 step2 : if few processes are blocked for the semaphore, wake up one process.

① Binary Semaphore



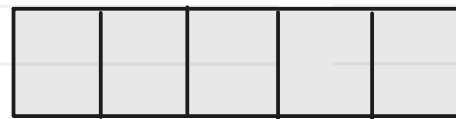
② Counting Semaphore




```
arr[5];
rear, front;
```

(Producer)

```
Thread1() {
    while(1) {
        P(es)
        P(bs)
        rear = (rear + 1) % size;
        arr[rear] = ↓ ;
        V(bs)
        V(fs)
    }
}
```



5
es

1
bs

0
fs

(Consumer)

```
Thread2() {
    while(1) {
        P(fs)
        P(bs)
        front = (front + 1) % size;
        Ⓣ = arr[front];
        V(bs)
        V(es)
    }
}
```

Mutual Exclusion

(one at a time)

- mutex is same as binary semaphore
- operations : `lock()` - before using resource
`unlock()` - after releasing resource
- process which will lock the mutex will become owner of that mutex.
- only owner can unlock the mutex

- infinite waiting of processes for a resource
- deadlock will occur in system when all four conditions will be true at a time
 1. Mutual Exclusion
 2. No preemption
 3. Hold & wait
 4. Circular wait



Prevention

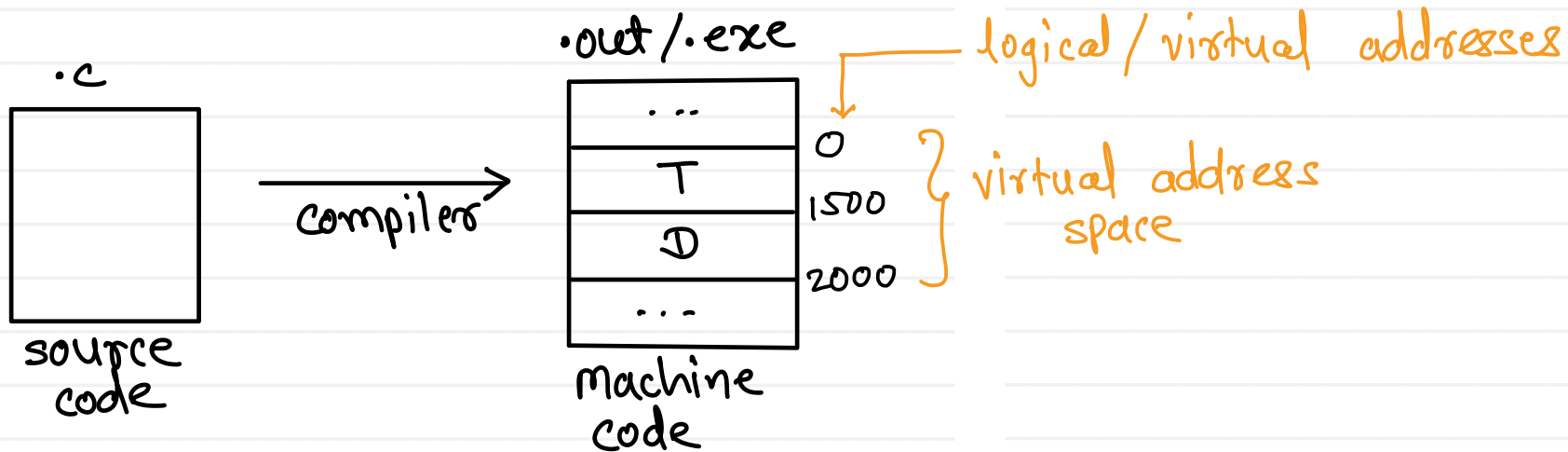
- while implementing OS, it is always ensured that one condition out of four will be always false

Avoidance

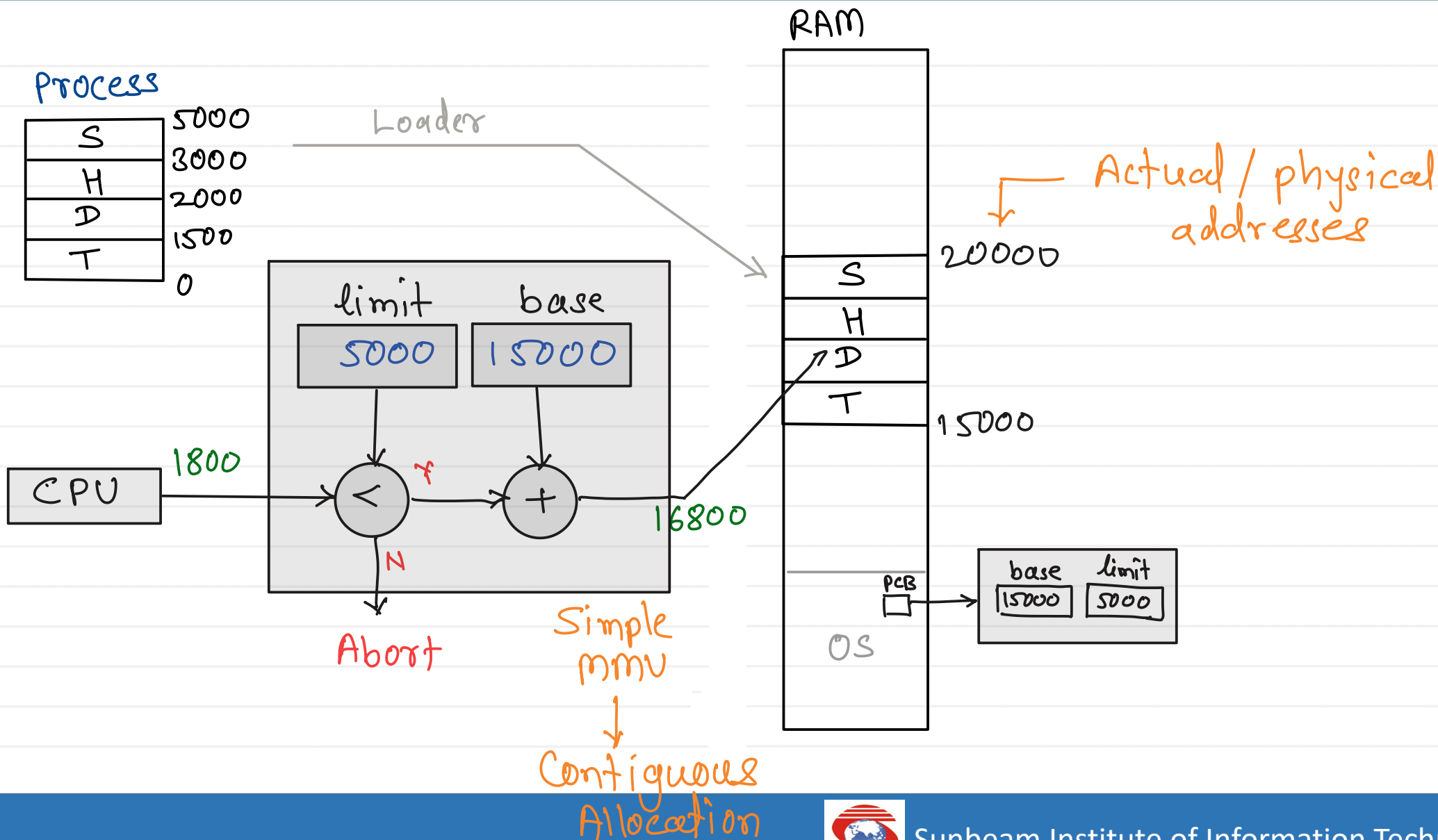
1. Resource allocation graph
2. Banker's algorithm

Recovery

- resource preemption
- forceful termination of process

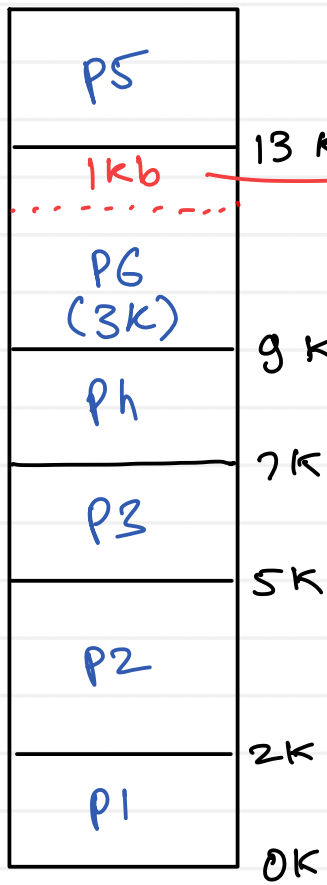


CPU always executes program in virtual address space.



Contiguous memory allocation

Fixed Partition



Internal fragmentation
- if process is not utilizing whole allocated space, some space is wasted

limitations :

1. size of process is restricted to the max size of partition
2. No. of processes are also limited to no. of partitions

Dynamic Partition

Free slot table

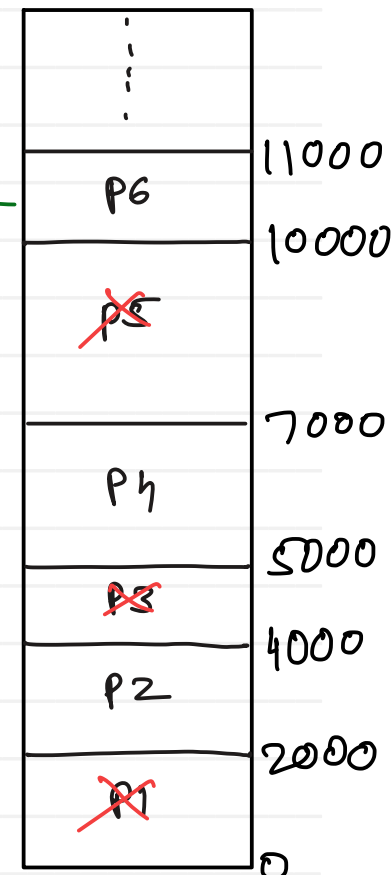
	limit	base
first fit	2000	0
best fit	1000	4000
worst fit	3000	7000
	2050	7950

size = 950 ✓
size = 4000 ✗

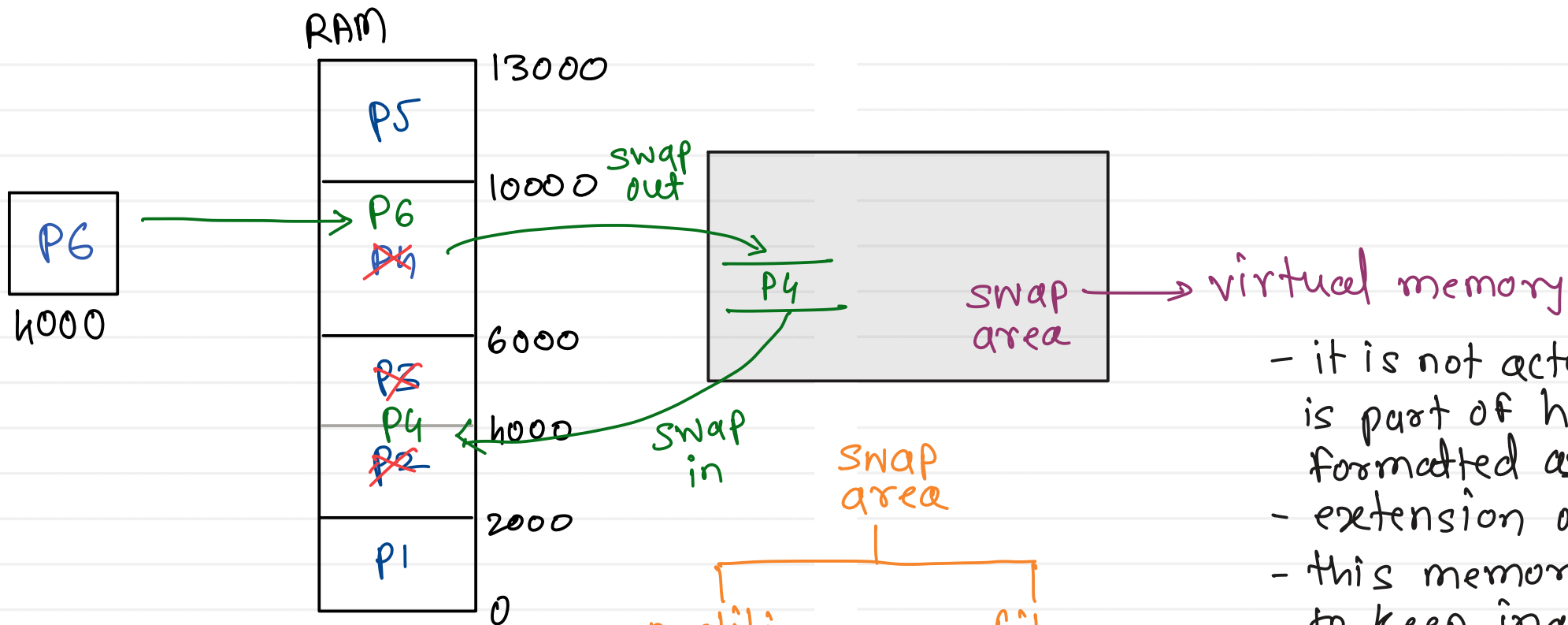
External fragmentation:
- due to unavailability of contiguous free space, process can not be loaded inside RAM

Compaction :

- moving processes inside RAM to create large contiguous free space.



Virtual memory



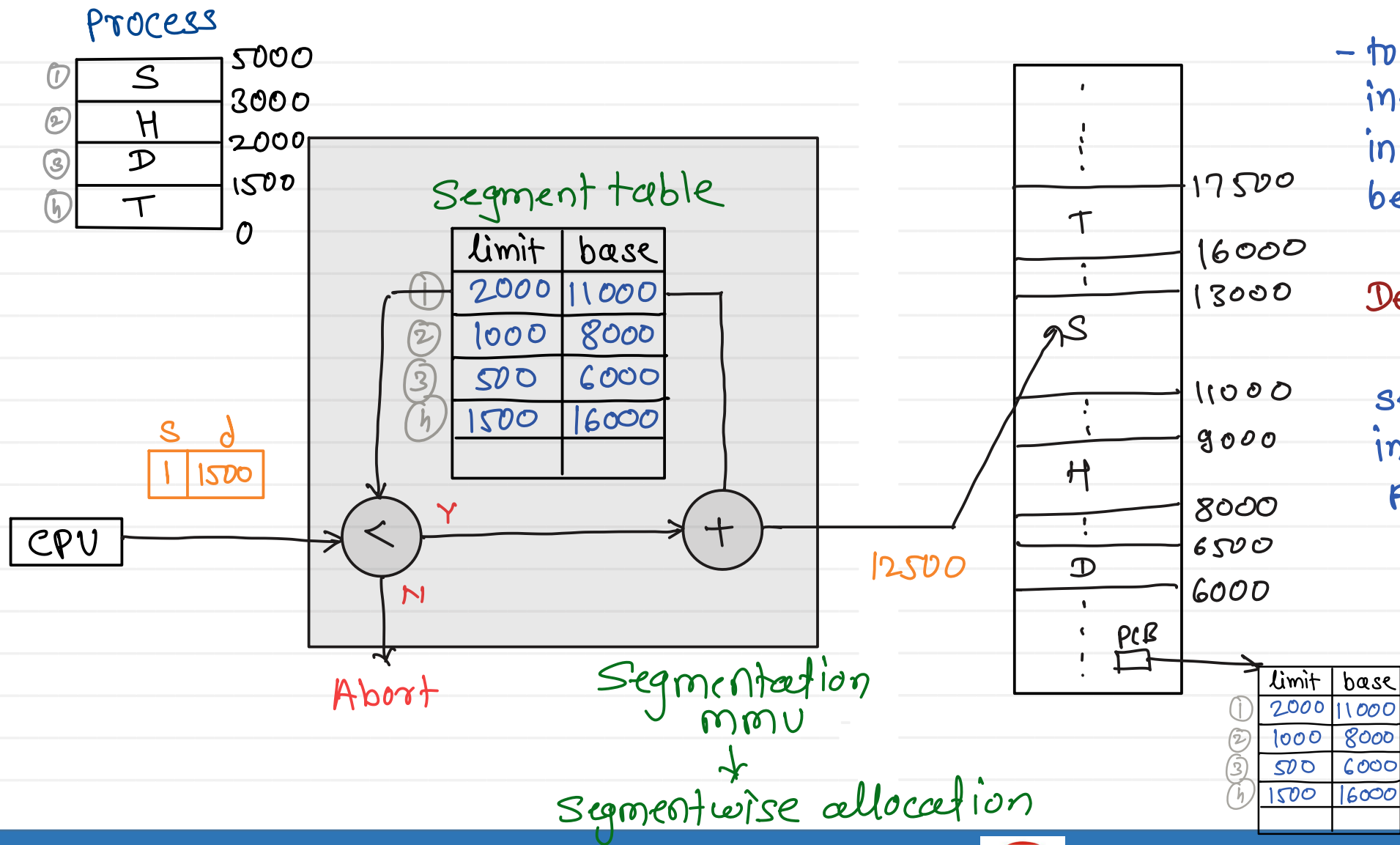
- it is not actual RAM, it is part of harddisk formatted as RAM
- extension of RAM
- this memory is used to keep inactive processes to create free space inside RAM

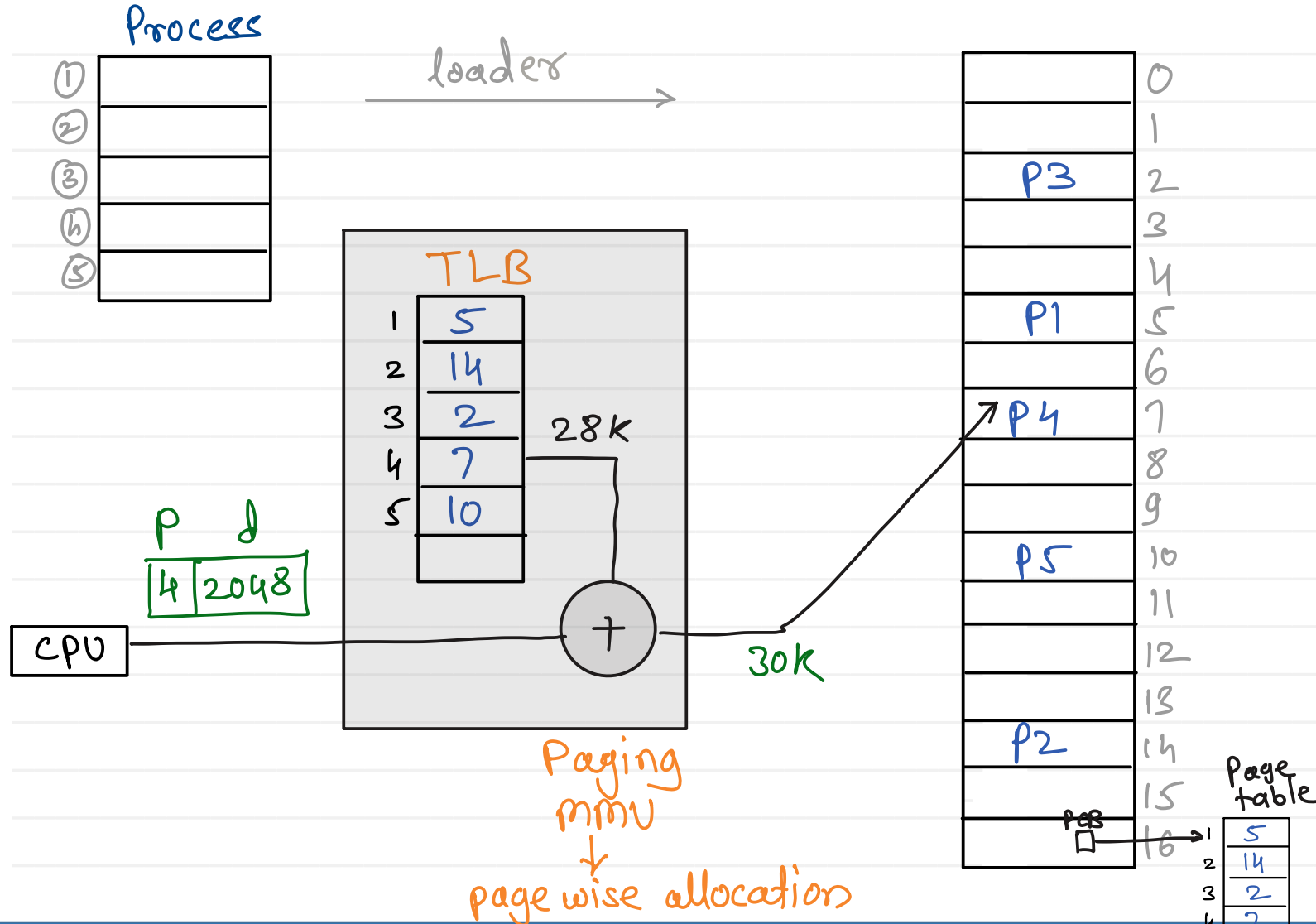
swap area

- partition
 - Linux
 - size = 2 * RAM size
- file
 - windows
 - c:\ → pagefile.sys

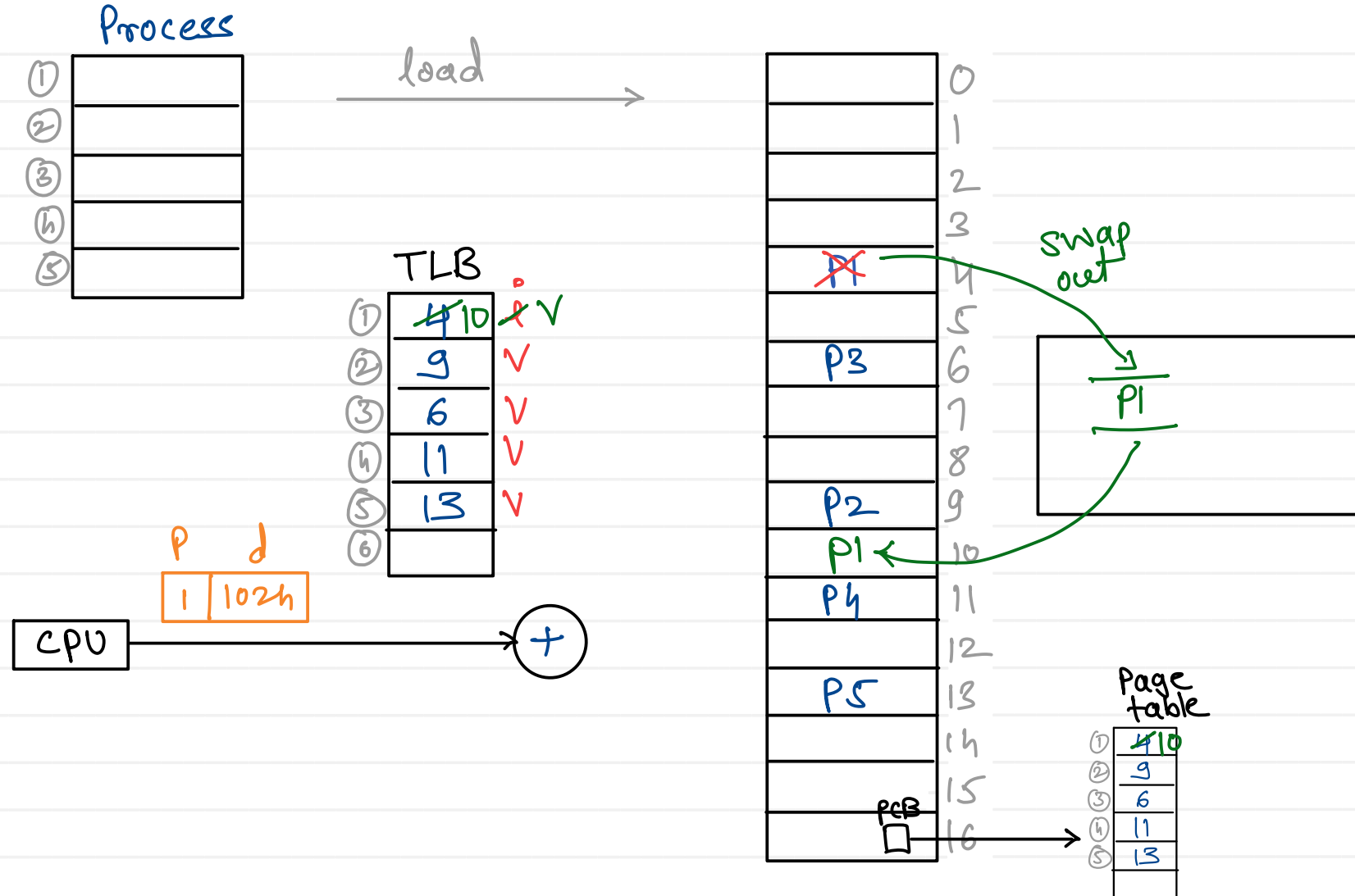
- process can not execute inside virtual memory

Segmentation MMU





- RAM is divided into fixed equal size partitions
frame/physical page
frame = 4096 bytes
- process is also divided into partitions of size equal to frame size
page/logical page



- When CPU tries to access address of some invalid entry page, "page fault" is generated in system.

- Whenever page fault is generated, page fault handler is called.

- page fault handler will first validate the address, find the free frame, swap in the page, modify entry of page table

`fork()` — to create a child process

`exec()` family — to load program into newly created process

Orphan process — whose parent terminates before it

Zombie process — terminated child process whose exit status is not read by its parent

- to avoid zombie process parent should read its exit status immediately.
- to read exit status parent uses `wait()` or `waitpid()` system calls